# TalentPulse HR & WFM System

SOLID Principles · Dependency Injection · Core Concepts

Spring Boot 3.2.5 · Java 17 · Spring Security + JWT

Spring Data JPA · Spring AOP · Maven

Complete Code-Level Analysis Report

| 187 | 22 | 5 | 21 |
|---|---|---|---|
| Java Files | Service Interfaces & Implementations | SOLID Principles Applied | DI Injection Points |

# Table of Contents

# 1. SOLID Principles in TalentPulse

SOLID is a set of five object-oriented design principles introduced by Robert C. Martin. Your TalentPulse project implements all five principles through its layered Spring Boot architecture. Below is a detailed, code-level analysis of each principle.

## 1.1 Single Responsibility Principle (SRP)

**Definition:** A class should have only one reason to change — it should do exactly one job.

TalentPulse enforces SRP rigorously through a clean four-layer architecture: **Controller** → **Service Interface** → **Service Implementation** → **Repository**. Each class owns a single concern and never bleeds into another layer.

### Where SRP is applied:

| Class / File | Single Responsibility |
|---|---|
| `EmployeeController.java` | HTTP request/response only — no business logic whatsoever |
| `EmployeeServiceImpl.java` | Employee CRUD business logic only |
| `EmployeeRepository.java` | Database access for Employee entity only |
| `JwtService.java` | JWT token creation and validation only |
| `JwtAuthenticationFilter.java` | Reads Bearer token, validates it, sets SecurityContext only |
| `GlobalExceptionHandler.java` | Centralised exception-to-HTTP-response mapping only |
| `LoggingAspect.java` | Execution-time logging for every service method — nothing else |
| `AuditAspect.java` | Writes AuditLog rows on create/update/delete — nothing else |
| `DataSeeder.java` | Seeds demo data on first startup — nothing else |
| `CorsConfig.java` | CORS origin configuration only |
| `SecurityConfig.java` | Spring Security filter chain configuration only |

### Code Evidence – EmployeeController.java:

```
@RestController
@RequestMapping("/employees")
public class EmployeeController {
```

```
private final EmployeeService employeeService; // only dependency
public EmployeeController(EmployeeService employeeService) {
this.employeeService = employeeService;
}
@PostMapping
public ResponseEntity<EmployeeDTO> create(@RequestBody EmployeeDTO dto) {
return new ResponseEntity<>(employeeService.create(dto), HttpStatus.CREATED);
}
// Only HTTP handling here — zero business/DB logic
}
```

## Code Evidence – LoggingAspect.java (single job: log timing):

```
@Aspect @Component
public class LoggingAspect {
@Around("execution(* com.talentpulse.serviceimpl.*.*(..))") // all service methods
public Object logExecutionTime(ProceedingJoinPoint joinPoint) throws Throwable {
long start = System.currentTimeMillis();
Object result = joinPoint.proceed();
long elapsed = System.currentTimeMillis() - start;
log.info("{} executed in {} ms", joinPoint.getSignature().toShortString(), elapsed);
return result;
}
}
```

# 1.2 Open / Closed Principle (OCP)

**Definition:** Software entities should be **open for extension** but **closed for modification**.

Every domain in TalentPulse is modelled as a **Service Interface + ServiceImpl** pair. The interface (contract) is closed — it never changes when new behaviour is needed. New behaviour is added by creating a new implementation or overriding methods without touching existing code.

## Where OCP is applied:

| Interface (Closed) | Implementation (Extension point) |
|---|---|
| `EmployeeService` | EmployeeServiceImpl — add new impl without changing interface |
| `LeaveApplicationService` | LeaveApplicationServiceImpl |
| `PayrollRunService` | PayrollRunServiceImpl — payroll rules extensible |
| `PayslipService` | PayslipServiceImpl — salary calc logic extensible |
| `NotificationService` | NotificationServiceImpl |
| `UserDetailsService (Spring)` | CustomUserDetailsService — Spring's contract, our extension |
| `OncePerRequestFilter (Spring)` | JwtAuthenticationFilter — Spring's contract, our extension |
| `WebMvcConfigurer (Spring)` | CorsConfig — Spring's contract, our extension |
| `CommandLineRunner (Spring)` | DataSeeder, RandomDataSeeder — pluggable startup runners |

## Code Evidence – Service Interface (never modified, always extended):

```
// EmployeeService.java — CLOSED contract
public interface EmployeeService {
List<EmployeeDTO> getAll();
EmployeeDTO getById(Long id);
EmployeeDTO create(EmployeeDTO dto);
EmployeeDTO update(Long id, EmployeeDTO dto);
void delete(Long id);
}

// EmployeeServiceImpl.java — OPEN for extension
@Service
public class EmployeeServiceImpl implements EmployeeService {
// Business logic lives here, interface contract untouched
}
```

**OCP via Spring Extension Points:** JwtAuthenticationFilter extends Spring's OncePerRequestFilter and CorsConfig implements WebMvcConfigurer — both are textbook OCP: Spring's framework code is closed;

your custom behaviour extends it.

# 1.3 Liskov Substitution Principle (LSP)

**Definition:** Objects of a subtype must be substitutable for objects of the supertype without altering the correctness of the program.

In TalentPulse every ServiceImpl is substitutable for its Service interface. Controllers depend only on the interface type — they are completely unaware of the concrete implementation. Swapping EmployeeServiceImpl for a different implementation would not require any change in EmployeeController.

## Code Evidence – LSP in action:

```
// Controller uses ONLY the interface type — LSP in practice
public class EmployeeController {
private final EmployeeService employeeService; // interface, not impl
// Works with ANY class that implements EmployeeService
}

// CustomUserDetailsService fully honours the UserDetailsService contract
@Service
public class CustomUserDetailsService implements UserDetailsService {
@Override
public UserDetails loadUserByUsername(String email) { ... }
// Throws UsernameNotFoundException exactly as contract requires
}
```

The **GlobalExceptionHandler** also demonstrates LSP: ResourceNotFoundException and BadRequestException both extend RuntimeException and are handled polymorphically — any handler that accepts RuntimeException works for both.

# 1.4 Interface Segregation Principle (ISP)

**Definition:** Clients should not be forced to depend on methods they do not use. Prefer many small, focused interfaces over one large one.

TalentPulse has **22 separate, purpose-specific service interfaces** — one per domain. No controller or class is forced to depend on unrelated methods.

## ISP across all 22 service interfaces:

| Service Interface | Methods (focused, not fat) |
|---|---|
| `EmployeeService` | getAll, getById, create, update, delete |
| `LeaveApplicationService` | getAll, getById, create, update, delete |
| `PayrollRunService` | getAll, getById, create, update, delete, processPayroll |

| | |
|---|---|
| `PayslipService` | getAll, getById, create, update, delete, generateForRun |
| `AttendanceRecordService` | getAll, getById, create, update, delete |
| `AppraisalCycleService` | getAll, getById, create, update, delete |
| `NotificationService` | getAll, getById, create, update, delete |
| `AuditLogService` | getAll, getById, create, update, delete |
| `JobRequisitionService` | getAll, getById, create, update, delete |
| `CandidateService` | getAll, getById, create, update, delete |
| `PromotionRecommendationService` | getAll, getById, create, update, delete |

Repository interfaces also follow ISP perfectly. EmployeeRepository only exposes findByDepartment_DepartmentId, findByReportingManager_EmployeeId, and findByStatus — domain-specific queries only, nothing unrelated.

# 1.5 Dependency Inversion Principle (DIP)

**Definition:** High-level modules should not depend on low-level modules. Both should depend on **abstractions** (interfaces). Abstractions should not depend on details; details should depend on abstractions.

DIP is the backbone of TalentPulse's architecture. Every Controller depends on a Service *interface*, every ServiceImpl depends on Repository *interfaces* — never on concrete classes. Spring's IoC container resolves and injects the concrete implementations at runtime.

## Code Evidence – Complete DIP chain:

```
// HIGH-LEVEL: Controller depends on abstraction
public class EmployeeController {
private final EmployeeService employeeService; // interface
}

// MID-LEVEL: ServiceImpl depends on repository abstractions
@Service
public class EmployeeServiceImpl implements EmployeeService {
private final EmployeeRepository employeeRepository; // interface
private final GradeStructureRepository gradeStructureRepository; // interface
private final DepartmentRepository departmentRepository; // interface
}

// LOW-LEVEL: JpaRepository is the abstraction Spring Data implements
public interface EmployeeRepository extends JpaRepository<Employee, Long> { }

// SecurityConfig depends on abstractions, not concrete classes
public class SecurityConfig {
private final JwtAuthenticationFilter jwtAuthFilter; // concrete (component)
private final CustomUserDetailsService userDetailsService; // Spring contract
}
```

| High-Level Module | Depends On (Abstraction) |
|---|---|
| `EmployeeController` | EmployeeService (interface) |
| `LeaveApplicationController` | LeaveApplicationService (interface) |
| `PayrollRunController` | PayrollRunService (interface) |
| `AuthController` | UserService, JwtService (services) |
| `EmployeeServiceImpl` | EmployeeRepository (JpaRepository) |
| `PayslipServiceImpl` | PayslipRepository, EmployeeRepository |
| `AuditAspect` | AuditLogRepository, UserRepository |

| | |
|---|---|
| `JwtAuthenticationFilter` | UserDetailsService (interface) |

# 2. Dependency Injection – Complete Map

TalentPulse uses exclusively Constructor Injection — the recommended Spring best practice. All dependencies are declared as private final fields and injected through the constructor, making classes immutable, easily testable, and free of hidden dependencies.

## Why Constructor Injection?

- **Immutability:** Fields declared final cannot be changed after construction.
- **Testability:** Dependencies passed in constructor are easy to mock in unit tests.
- **Fail-fast:** Missing beans cause startup failure, not a NullPointerException at runtime.
- **No @Autowired annotation needed** on single-constructor classes (Spring 4.3+).

## Controller Layer — Injection Points:

| Controller Class | Injected Dependencies (all interfaces) |
|---|---|
| `EmployeeController` | EmployeeService |
| `LeaveApplicationController` | LeaveApplicationService |
| `LeaveBalanceController` | LeaveBalanceService |
| `LeaveTypeController` | LeaveTypeService |
| `PayrollRunController` | PayrollRunService |
| `PayslipController` | PayslipService |
| `AttendanceRecordController` | AttendanceRecordService |
| `AppraisalCycleController` | AppraisalCycleService |
| `GoalSheetController` | GoalSheetService |
| `PromotionRecommendationController` | PromotionRecommendationService |
| `CandidateController` | CandidateService |
| `JobRequisitionController` | JobRequisitionService |
| `InterviewRoundController` | InterviewRoundService |
| `OfferLetterController` | OfferLetterService |
| `DepartmentController` | DepartmentService |

| | |
|---|---|
| `EmployeeController` | EmployeeService |
| `GradeStructureController` | GradeStructureService |
| `SalaryStructureController` | SalaryStructureService |
| `NotificationController` | NotificationService |
| `HRReportController` | HRReportService |
| `AuditLogController` | AuditLogService |
| `UserController` | UserService |
| `AuthController` | AuthenticationManager, UserService, UserRepository, JwtService, UserDetailsService |

## Service Implementation Layer — Injection Points:

| ServiceImpl Class | Injected Repositories / Services |
|---|---|
| `EmployeeServiceImpl` | EmployeeRepository, GradeStructureRepository, DepartmentRepository |
| `LeaveApplicationServiceImpl` | LeaveApplicationRepository, EmployeeRepository, LeaveTypeRepository |
| `PayrollRunServiceImpl` | PayrollRunRepository, UserRepository, PayslipRepository, EmployeeRepository, PayslipService |
| `PayslipServiceImpl` | PayslipRepository, PayrollRunRepository, EmployeeRepository, SalaryStructureRepository, GradeStructureRepository |
| `LeaveBalanceServiceImpl` | LeaveBalanceRepository, EmployeeRepository, LeaveTypeRepository |
| `AttendanceRecordServiceImpl` | AttendanceRecordRepository, EmployeeRepository |
| `AppraisalCycleServiceImpl` | AppraisalCycleRepository |
| `GoalSheetServiceImpl` | GoalSheetRepository, EmployeeRepository, AppraisalCycleRepository |
| `PromotionRecommendationServiceImpl` | PromotionRecommendationRepository, EmployeeRepository |
| `CandidateServiceImpl` | CandidateRepository, JobRequisitionRepository |
| `InterviewRoundServiceImpl` | InterviewRoundRepository, CandidateRepository, EmployeeRepository |
| `OfferLetterServiceImpl` | OfferLetterRepository, CandidateRepository |

| | |
|---|---|
| `NotificationServiceImpl` | NotificationRepository, UserRepository |
| `AuditLogServiceImpl` | AuditLogRepository |
| `UserServiceImpl` | UserRepository, PasswordEncoder |
| `HRReportServiceImpl` | HRReportRepository, EmployeeRepository |

## Security / AOP / Config Layer — Injection Points:

| Class | Injected Dependencies |
|---|---|
| `SecurityConfig` | JwtAuthenticationFilter, CustomUserDetailsService |
| `JwtAuthenticationFilter` | JwtService, UserDetailsService |
| `CustomUserDetailsService` | UserRepository |
| `AuditAspect` | AuditLogRepository, UserRepository |
| `LoggingAspect` | (none — uses AOP joinpoint only) |
| `DataSeeder` | UserRepository, EmployeeRepository, DepartmentRepository, GradeStructureRepository, LeaveTypeRepository, SalaryStructureRepository, AppraisalCycleRepository, PasswordEncoder |

## Canonical Constructor Injection Pattern (used everywhere):

```
@Service
public class PayrollRunServiceImpl implements PayrollRunService {

private final PayrollRunRepository payrollRunRepository; // final = immutable
private final UserRepository userRepository;
private final PayslipRepository payslipRepository;
private final EmployeeRepository employeeRepository;
private final PayslipService payslipService;

// Spring injects all dependencies here automatically
public PayrollRunServiceImpl(PayrollRunRepository payrollRunRepository,
UserRepository userRepository,
PayslipRepository payslipRepository,
EmployeeRepository employeeRepository,
PayslipService payslipService) {
this.payrollRunRepository = payrollRunRepository;
this.userRepository = userRepository;
this.payslipRepository = payslipRepository;
this.employeeRepository = employeeRepository;
this.payslipService = payslipService;
}
}
```

# 3. Core Concepts & Technologies Used

## 3.1 Spring Boot & Auto-Configuration

TalentPulseApplication.java carries @SpringBootApplication which triggers auto-configuration, component scanning, and the embedded Tomcat server. Spring Boot 3.2.5 on Java 17.

- @SpringBootApplication on TalentPulseApplication.java — single entry point
- Auto-configures JPA, Security, Web, AOP, Validation from pom.xml dependencies
- application.properties drives datasource, JWT secret, and server config

## 3.2 Spring MVC – REST Controllers

22 @RestController classes handle all REST endpoints. Each uses @RequestMapping, @GetMapping, @PostMapping, @PutMapping, @DeleteMapping and returns ResponseEntity or DTO directly.

- @RestController = @Controller + @ResponseBody — JSON by default
- @RequestMapping("/employees") — base path per resource
- @PathVariable, @RequestBody — parameter binding
- ResponseEntity<Void> with noContent().build() for DELETE responses

## 3.3 Spring Data JPA – Repository Pattern

20 repository interfaces extend JpaRepository<Entity, Long>. Spring Data generates all CRUD SQL at runtime — no boilerplate DAO code. Custom derived queries are declared by method name.

- JpaRepository provides findAll, findById, save, delete out of the box
- EmployeeRepository: findByDepartment_DepartmentId, findByStatus (derived queries)
- @Entity, @Table, @Id, @GeneratedValue — JPA entity mappings
- @ManyToOne with FetchType.LAZY — avoids N+1 query problems

## 3.4 Spring Security + JWT Authentication

Stateless JWT-based security. Every request passes through JwtAuthenticationFilter which validates the Bearer token and sets the SecurityContext. Role-based access control via hasRole() rules in SecurityConfig.

- JwtService: token generation (HS256), validation, claims extraction using jjwt 0.11.5
- JwtAuthenticationFilter extends OncePerRequestFilter — runs once per HTTP request
- CustomUserDetailsService implements UserDetailsService — loads user by email
- SecurityConfig: CSRF disabled, STATELESS session, @EnableMethodSecurity
- BCryptPasswordEncoder for password hashing
- Role-based rules: /users/** → ADMIN only; /auth/** → public

## 3.5 Spring AOP – Cross-Cutting Concerns

Two @Aspect components cleanly separate cross-cutting concerns from business logic, eliminating the need for logging/auditing code inside every service method.

- LoggingAspect: @Around on all serviceimpl methods — measures and logs execution time
- AuditAspect: @AfterReturning on create/update/delete — writes AuditLog to DB
- AuditAspect uses @Pointcut to define reusable pointcut expression
- Reads current user from Spring SecurityContextHolder for audit trail
- Both aspects use constructor-injected repositories — consistent with project DI style

## 3.6 DTO Pattern (Data Transfer Objects)

22 DTO classes act as the API contract. They decouple the HTTP layer from JPA entities, preventing accidental exposure of internal fields, lazy-load proxies, or sensitive data.

- EmployeeDTO, LeaveApplicationDTO, PayrollRunDTO … one DTO per domain entity
- toDto() and toEntity() mapping methods inside each ServiceImpl
- AuthRequest, AuthResponse, RegisterRequest — dedicated auth DTOs
- OfferLetterDTO, PayslipDTO carry computed/derived fields not on the entity

## 3.7 Global Exception Handling

@RestControllerAdvice in GlobalExceptionHandler centralises all exception-to-response mapping, keeping controllers free of try/catch blocks.

- ResourceNotFoundException → 404 NOT FOUND
- BadRequestException → 400 BAD REQUEST
- MethodArgumentNotValidException → 400 with field-level error map
- Generic Exception → 500 INTERNAL SERVER ERROR
- Every error response includes timestamp, status, error, message fields

## 3.8 Enums for Type Safety

20 enum classes replace magic strings throughout the codebase, making state transitions compile-time safe and self-documenting.

- EmployeeStatus: ACTIVE, PROBATION, RESIGNED, EXITED
- PayrollStatus: DRAFT, PROCESSING, COMPLETED, CANCELLED
- LeaveStatus: PENDING, APPROVED, REJECTED
- Role: ADMIN, HR, MANAGER, EMPLOYEE
- Stored as @Enumerated(EnumType.STRING) in the database

## 3.9 CommandLineRunner – Data Seeding

DataSeeder and RandomDataSeeder implement CommandLineRunner to populate demo data on first startup. @Order(1) / @Order(2) controls execution sequence.

- DataSeeder seeds reference data: departments, grades, leave types, users
- RandomDataSeeder seeds random employees, payroll runs, attendance records
- Checks if data already exists before seeding — idempotent
- Uses @Value to read seeding-enabled flag from application.properties

## 3.10 CORS Configuration

CorsConfig implements WebMvcConfigurer to allow the React frontend (localhost:3000 / 5173) to call the API. Permits all HTTP methods and headers.

- Registered via addCorsMappings() — applies globally to all endpoints
- Also configured inline in SecurityConfig for Spring Security filter chain
- Allowed origins: http://localhost:3000 and http://localhost:5173

# 4. Project Architecture Overview

| Package | Classes | Role |
|---|---|---|
| `com.talentpulse.controller` | 22 Controllers | HTTP in/out, delegates to Service interface |
| `com.talentpulse.service` | 22 Service Interfaces | Contract/abstraction for business logic |
| `com.talentpulse.serviceimpl` | 22 ServiceImpl classes | Actual business logic, injected with repos |
| `com.talentpulse.repository` | 20 Repository Interfaces | Spring Data JPA — DB access layer |
| `com.talentpulse.entity` | 20 JPA Entities | @Entity mapped to DB tables |
| `com.talentpulse.dto` | 22 DTO classes | API request/response contracts |
| `com.talentpulse.enums` | 20 Enums | Type-safe status/category values |
| `com.talentpulse.exception` | 3 classes | Custom exceptions + GlobalExceptionHandler |
| `com.talentpulse.security` | 3 classes | JWT filter, JWT service, UserDetails loader |
| `com.talentpulse.config` | 2 classes | SecurityConfig, CorsConfig |
| `com.talentpulse.aop` | 2 Aspects | LoggingAspect, AuditAspect |
| `com.talentpulse.bootstrap` | 2 classes | DataSeeder, RandomDataSeeder |

## Request Flow:

```
HTTP Request
■■■ JwtAuthenticationFilter (validates Bearer token, sets SecurityContext)
■■■ SecurityConfig (checks role-based access rules)
■■■ @RestController (maps URL → method, reads @PathVariable/@RequestBody)
■■■ Service Interface (DIP boundary)
■■■ ServiceImpl (business logic)
■■■ Repository Interface → Spring Data JPA → Database
■■■ Other Services (e.g. PayrollRunService → PayslipService)
↕
AOP Aspects intercept transparently:
LoggingAspect (timing) + AuditAspect (audit trail)
■■■ ResponseEntity / DTO
■■■ GlobalExceptionHandler (if exception thrown at any layer)
```

# 5. Summary

| SOLID Principle | Applied In | Key Files |
|---|---|---|
| **S – Single Responsibility** | Each class has exactly one job | `All 22 controllers, 22 service impls, 2 AOP aspects, GlobalExceptionHandler` |
| **O – Open / Closed** | 22 Service interface+impl pairs; Spring extension points | `All *Service.java interfaces; JwtAuthenticationFilter, CorsConfig` |
| **L – Liskov Substitution** | ServiceImpl always substitutable for Service interface | `All ServiceImpl classes; CustomUserDetailsService` |
| **I – Interface Segregation** | 22 focused service interfaces; 20 focused repository interfaces | `All *Service.java; All *Repository.java` |
| **D – Dependency Inversion** | Controllers→interfaces, impls→repo interfaces | `All controllers and serviceimpl classes` |

Your TalentPulse project demonstrates strong, professional adherence to SOLID principles. Constructor injection is used consistently across all 21+ injection points. The clean separation of Controller → Service Interface → ServiceImpl → Repository makes the codebase highly maintainable, testable, and extensible. The addition of Spring AOP for logging and auditing is a particularly strong architectural choice that keeps business logic completely clean of cross-cutting concerns.

*Generated by automated code analysis of talentpulse-backend · Spring Boot 3.2.5 · Java 17 · 187 Java files analysed*